CORRIGÉ · PROBLÈME · ANNALES MP

Relation d'ordre et tri topologique

Énoncé

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

1. Écrire `check_reflexivity (r:order) : bool` dans le paradigme **impératif**.
2. Écrire `check_reflexivity_bis (r:order) : bool` dans le paradigme **fonctionnel**.
3. Écrire `check_antisymmetry (r:order) : bool`.
4. Écrire `check_transitivity (r:order) : bool`.
5. Écrire `is_order (r:order) : bool` à partir de 1, 3 et 4.

Le graphe de l'ordre a pour arcs les (v_i, v_j) tels que $v_i \trianglelefteq v_j$; le degré sortant d'un sommet est le nombre d'arcs qui en partent.

1. Écrire `outdegrees (r:order) : int array` (degrés sortants).
2. Écrire `argmin (d:int array) : int` (un indice d'une valeur minimale).

Un tri topologique est une permutation σ de $[\![0, n-1]\!]$ telle que $v_i \trianglelefteq v_j$ entraîne $\sigma(i) \leqslant \sigma(j)$; v_i est alors le sommet de numéro $\sigma(i)$.

1. Montrer qu'il existe une constante δ_0 telle que, pour tout tri topologique σ, le sommet de numéro $n-1$ est de degré sortant δ_0 et les autres de degré sortant $\geqslant \delta_0$.
2. Écrire `topological_sort (r:order) : int array` qui attribue les numéros par ordre décroissant en s'appuyant sur la question 8.
3. Énoncer des invariants qui prouvent la correction de `topological_sort`.
4. Complexité en temps de `topological_sort` en fonction de n ; comment la qualifier par rapport à la taille de l'entrée ?

Décoder l'énoncé

Ce que l'énoncé te donne.

- La matrice est de taille $n \times n$: lire toute l'entrée coûte déjà n^2.
- Réflexivité : chaque sommet a une boucle, donc un degré sortant $\geqslant 1$.

Ce qu'on te demande. Vérifier les axiomes d'un ordre, puis numéroter les sommets en retirant à chaque fois un élément maximal.

Corrigé

Question 1

Étape 1. Q1. Réflexivité, version impérative

```
let check_reflexivity (r:order) : bool =
  let n = Array.length r in
  let ok = ref true in
  for i = 0 to n - 1 do
    if not r.(i).(i) then ok := false   (* v_i n'est pas en relation avec lui-même *)
  done;
  !ok
```

Style impératif : variable mutable (référence) et boucle. Complexité $O(n)$.

Question 2

Étape 1. Q2. Réflexivité, version fonctionnelle

```
let check_reflexivity_bis (r:order) : bool =
  let n = Array.length r in
  let rec from i = i >= n || (r.(i).(i) && from (i + 1)) in
  from 0
```

Aucune variable mutable : la fonction récursive `from i` vaut « tous les $r_{k,k}$ pour $k \geqslant i$ sont vrais ». Le `&&` paresseux arrête dès le premier échec. Variante tout aussi fonctionnelle : `Array.for_all (fun b -> b) (Array.mapi (fun i ligne -> ligne.(i)) r)`.

Question 3

Étape 1. Q3. Antisymétrie

```
let check_antisymmetry (r:order) : bool =
  let n = Array.length r in
  let ok = ref true in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if i <> j && r.(i).(j) && r.(j).(i) then ok := false
    done
  done;
  !ok
```

Comme les v_i sont distincts, $v_i = v_j$ équivaut à $i = j$: l'antisymétrie interdit d'avoir les deux relations pour $i \neq j$. Complexité $O(n^2)$.

Question 4

Étape 1. Q4. Transitivité

```
let check_transitivity (r:order) : bool =
  let n = Array.length r in
  let ok = ref true in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if r.(i).(j) then
        for k = 0 to n - 1 do
          if r.(j).(k) && not r.(i).(k) then ok := false   (* contre-exemple *)
        done
    done
  done;
  !ok
```

On teste tous les triplets (i, j, k) : complexité $O(n^3)$.

Question 5

Étape 1. Q5. Est-ce un ordre ?

```
let is_order (r:order) : bool =
  check_reflexivity r && check_antisymmetry r && check_transitivity r
```

Question 6

Étape 1. Q6. Degrés sortants

```
let outdegrees (r:order) : int array =
  let n = Array.length r in
  let d = Array.make n 0 in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if r.(i).(j) then d.(i) <- d.(i) + 1   (* arc (v_i, v_j) *)
    done
  done;
  d
```

Les arcs issus de v_i sont les (v_i, v_j) avec $v_i \trianglelefteq v_j$, boucle (v_i, v_i) comprise. Complexité $O(n^2)$.

Question 7

Étape 1. Q7. Indice d'un minimum

```
let argmin (d:int array) : int =
  let best = ref 0 in
  for j = 1 to Array.length d - 1 do
    if d.(j) < d.(!best) then best := j
  done;
  !best
```

On suppose `d` non vide. Invariant : après le tour j, `d.(!best)` est le minimum de `d.(0..j)`.

Question 8

Étape 1. Q8. Le dernier sommet a un degré sortant de 1

On montre que $\delta_0 = 1$ convient.

Tous les degrés sont $\geqslant 1$: par réflexivité, chaque v_i a la boucle (v_i, v_i).

Le sommet de numéro $n-1$ **a un degré exactement** 1 : soit $u = v_{\sigma^{-1}(n-1)}$ et soit un arc (u, v_j), c'est-à-dire $u \trianglelefteq v_j$. Par définition du tri topologique, $\sigma(j) \geqslant \sigma(\sigma^{-1}(n-1)) = n-1$, donc $\sigma(j) = n-1$ (c'est le plus grand numéro), donc $v_j = u$ (σ est une bijection). Le seul arc issu de u est sa boucle.

> $\delta_0 = 1$: le sommet de numéro $n-1$ est de degré sortant 1 (c'est un élément maximal), les autres de degré $\geqslant 1$.

Conséquence utile : un sommet de degré sortant minimal (égal à 1) est un élément maximal ; c'est lui qu'on numérote en dernier.

Question 9

Étape 1. Q9. Le tri topologique

Idée : d'après Q8 appliquée à l'ensemble des sommets restants, un sommet de degré sortant minimal (dans le graphe restant) est maximal. On lui donne le plus grand numéro disponible, on le retire, et on recommence. Retirer v diminue de 1 le degré de chaque sommet restant $u \neq v$ tel que $u \trianglelefteq v$; on met le degré de v à `max_int` pour ne plus le choisir.

```
let topological_sort (r:order) : int array =
  let n = Array.length r in
  let d = outdegrees r in              (* degrés sortants dans le graphe restant *)
  let sigma = Array.make n 0 in
  for k = n - 1 downto 0 do
    let v = argmin d in                (* sommet maximal parmi les restants (Q8) *)
    sigma.(v) <- k;                    (* il reçoit le plus grand numéro libre *)
    d.(v) <- max_int;                  (* on le retire *)
    for u = 0 to n - 1 do
      if d.(u) <> max_int && r.(u).(v) then d.(u) <- d.(u) - 1   (* arc (u,v) supprimé *)
    done
  done;
  sigma
```

Question 10

Étape 1. Q10. Invariants de correction

Notons R_k l'ensemble des sommets numérotés avant le tour k (au départ, $R_{n-1} = \emptyset$). Au début du tour k :

- **(I1)** σ réalise une bijection de R_k sur $[\![k+1, n-1]\!]$;
- **(I2)** pour tout $u \notin R_k$: `d.(u)` $=$ card$\{w \notin R_k : v_u \trianglelefteq v_w\}$ (degré sortant dans le graphe restant), et `d.(u)` $=$ `max_int` pour $u \in R_k$;
- **(I3)** si $v_i \trianglelefteq v_j$ et $i \in R_k$, alors $j \in R_k$ et $\sigma(i) \leqslant \sigma(j)$ (aucun sommet numéroté n'est en dessous d'un sommet restant).

Initialisation : $R = \emptyset$, `d` $=$ degrés de Q6 : vrai.

Conservation : le sommet restant est un ensemble ordonné (restriction de l'ordre). D'après Q8 appliquée à lui, le minimum des degrés restants vaut 1 et le sommet v choisi est maximal parmi les restants : si $v \trianglelefteq w$ avec w restant, alors $w = v$. On lui donne le numéro k :

- (I1) : σ est une bijection de $R_k \cup \{v\}$ sur $[\![k, n-1]\!]$;
- (I3) : si $v \trianglelefteq w$, alors soit $w = v$, soit $w \in R_k$, de numéro $> k$; pour les anciens sommets numérotés, rien ne change ;
- (I2) : retirer v enlève exactement un arc (u, v) à chaque u restant tel que $v_u \trianglelefteq v$; c'est la mise à jour faite.

Conclusion : après le tour $k = 0$, $R = V$, σ est une permutation de $[\![0, n-1]\!]$ (I1) et respecte l'ordre (I3) : c'est un tri topologique.

Question 11

Étape 1. Q11. Complexité

`outdegrees` coûte $O(n^2)$. Puis n tours, chacun avec un `argmin` en $O(n)$ et une mise à jour en $O(n)$: $O(n^2)$.

Complexité : $\Theta(n^2)$.

L'entrée `r` occupe n^2 cases : la complexité est **linéaire en la taille de l'entrée**. On ne peut pas faire mieux, puisqu'il faut au moins lire la matrice.

Le piège

En Q9, ne pas mettre à jour les degrés après avoir retiré un sommet : le deuxième sommet choisi ne serait plus forcément maximal parmi les restants.

La méthode

Tri topologique « par la fin » : retirer à chaque étape un élément maximal (degré sortant 1 grâce à la réflexivité), en maintenant les degrés du graphe restant.

Rappels de cours utilisés

- **Relation d'ordre.** Réflexive ($x \trianglelefteq x$), antisymétrique ($x \trianglelefteq y$ et $y \trianglelefteq x$ donnent $x = y$), transitive ($x \trianglelefteq y \trianglelefteq z$ donne $x \trianglelefteq z$). Une **chaîne** est une partie où deux éléments sont toujours comparables ; une **antichaîne**, une partie où deux éléments distincts ne le sont jamais.
- **Invariant de boucle.** Propriété vraie avant la boucle, conservée par chaque tour ; à la sortie, avec la condition d'arrêt, elle donne le résultat. C'est l'outil standard pour prouver la correction d'un programme itératif.